FH University of Applied Sciences TECHNIKUM WIEN

Artificial Intelligence

Modul 2: Logik

Informatik

Motivation

- Warum logikbasierte AI?
  → Datenbasierte AI kann nicht alles!

- Wissenschaftliche Analysen – auch aus der Industrie – zeigen, dass LLMs bzw. datenbasierte AI bestimmte Aufgaben nicht gut bewältigen. Siehe z.B.:
  „The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity“ (Shojaee et al., 2025)
  https://machinelearning.apple.com/research/illusion-of-thinking

© FH Technikum Wien

Lernergebnisse und Inhalt

Modul 2: „***Wissensbasen mittels der Formalismen von Aussagen- und Prädikatenlogik aufzubauen und Inferenzregeln darauf anzuwenden, um logische Schlussfolgerungen abzuleiten.***“

1. Aussagenlogik
2. Prädikatenlogik
3. Logikorientierte Programmierung

© FH Technikum Wien

1. Aussagenlogik

© FH Technikum Wien

Was versteht man unter Aussagenlogik?

- Eine mathematische Sprache, um Wissen in strukturierter und maschinenlesbarer Form darzustellen

- Aussagen werden über logische Operationen verknüpft

wenn es regnet ist die Straße nass.

$$\textit{es regnet} \Rightarrow \textit{die Straße ist nass.}$$

- Vorteil der 2. Schreibweise: maschinenlesbarer und Struktur unabhängig vom Inhalt von Basisaussagen

© FH Technikum Wien

Syntax

Definition 2.1

Sei $Op = \{\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow, (,)\}$ die Menge der logischen Operatoren und Σ eine Menge von Symbolen. Die Mengen Op, Σ und $\{w, f\}$ seien paarweise disjunkt. Σ heißt **Signatur** und seine Elemente sind die **Aussagevariablen** . Die Menge der Aussagenlogischen Formeln wird nun rekursiv definiert:

- w und f sind (atomare) Formeln.
- Alle Aussagevariablen, das heißt alle Elemente von Σ sind (atomare) Formeln.
- Sind A und B Formeln, so sind auch $\neg A$, (A), $A \wedge B$, $A \vee B$, $A \Rightarrow B$, $A \Leftrightarrow B$ Formeln.

Definition 2.2

Man liest die Operatoren und Symbole folgendermaßen:

w: „wahr“

f: „falsch“

$\neg A$: „nicht A“ (Negation)

$A \wedge B$: „A und B“ (Konjunktion)

$A \vee B$: „A oder B“ (Disjunktion)

$A \Rightarrow B$: „wenn A dann B“ (Implikation [1])

$A \Leftrightarrow B$: „A genau dann, wenn B“ (Äquivalenz)

Zudem gilt (Abschlussbedingung): *Formeln werden nur über diese Regeln gebildet, d.h. Ausdrücke die nicht über diese rekursive Definition gebildet werden können, sind nicht zulässig (etwa „A B“).*

© FH Technikum Wien

Möglichkeiten dieser Syntax

- Durch Definition unendlich viele Formeln möglich.

- Beispiel: gegeben $\Sigma = \{A, B, C\}$
  Syntaktisch korrekt: $(((A)) \vee B)$
  aber auch:

$$A \wedge B, \quad A \wedge B \wedge C, \quad A \wedge A \wedge A, \quad C \wedge B \vee A, \quad (\neg A \wedge B) \Rightarrow (\neg C \vee A)$$

→ Allerdings ist das nur das syntaktische Gebilde
Es fehlt noch die Semantik (=Bedeutung)!

© FH Technikum Wien

Semantik

- Zwei Wahrheitswerte: wahr und falsch
- Ist die Formel $A \wedge B$ wahr?
  Hängt von den Werten von A und B ab.
- Beispiel: A = „es regnet“, B = „es ist kalt“
  Ob „regnet UND kalt“ gilt,
  hängt von den Werten von A bzw. B ab.

→ Aussagevariablen müssen Wahrheitswerte zugeordnet werden, die Zuständen der Welt entsprechen

© FH Technikum Wien

Semantik

Definition 2.3
Eine Abbildung $I : \Sigma \rightarrow \{w, f\}$, die jeder Aussagevariablen einen Wahrheitswert zuordnet, heißt **Belegung** oder **Interpretation** oder auch **Welt**.

- → Jede aussagenlogische Formel mit n Variablen besitzt 2^n verschiedene Belegungen (bzw. Welten).
- → Wahrheitstabelle definiert resultierende Werte für die erlaubten Basisoperatoren

© FH Technikum Wien

Wahrheitstabelle für logischer Operatoren

Über 2 Variablen (A, B) sind $2^2 = 4$ Interpretationen möglich

A	B	(A)	$\neg A$	$A \wedge B$	$A \vee B$	$A \Rightarrow B$	$A \Leftrightarrow B$
w	w	w	f	w	w	w	w
w	f	w	f	f	w	f	f
f	w	f	w	f	w	w	f
f	f	f	w	f	f	w	w

- Der Wert einer atomaren Formel (=eine alleinstehende Variable) wird direkt durch die Interpretation bestimmt
- Zur Ermittlung der Wahrheitswerte für komplexe Formeln ist Rangfolge der logischen Operatoren notwendig
  → zuerst Klammerung, dann folgende Reihenfolge:

$$\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$$

© FH Technikum Wien

Äquivalenz – Syntax vs. Semantik

Definition 2.4
Zwei Formeln F und G heißen semantisch äquivalent, wenn sie für alle Belegungen die gleichen Wahrheitswerte annehmen. Man schreibt $F \equiv G$.

- → Im Unterschied zur syntaktischen Äquivalenz
- → Semantik = Metasprache (nämlich die natürliche Sprache) über die Objektsprache (die Logik)

Definition 2.5
Eine Formel heißt

- **erfüllbar** , falls sie bei mindestens einer Belegung wahr ist.
- **allgemeingültig** oder **wahr** , falls sie bei allen Belegungen wahr ist. Wahre Formeln heißen auch **Tautologien** .
- **unerfüllbar** , falls sie bei keiner Belegung wahr ist.

Jede erfüllende Belegung einer Formel heißt **Modell** der Formel.

© FH Technikum Wien

In der Praxis wichtige Äquivalenzen

Satz 1

Die Operationen $\wedge, \vee$ sind kommutativ und assoziativ und folgende Äquivalenzen sind allgemeingültig:

$$\begin{aligned}
\neg A \vee B &\Leftrightarrow A \Rightarrow B && \text{(Implikation)}\\
A \Rightarrow B &\Leftrightarrow \neg B \Rightarrow \neg A && \text{(Kontraposition)}\\
(A \Rightarrow B) \wedge (B \Rightarrow A) &\Leftrightarrow (A \Leftrightarrow B) && \text{(Äquivalenz)}\\
\neg(A \wedge B) &\Leftrightarrow \neg A \vee \neg B && \text{(De Morgan'sche Regeln}\\
\neg(A \vee B) &\Leftrightarrow \neg A \wedge \neg B\\
A \vee (B \wedge C) &\Leftrightarrow (A \vee B) \wedge (A \vee C) && \text{(Distributivgesetze)}\\
A \wedge (B \vee C) &\Leftrightarrow (A \wedge B) \vee (A \wedge C)\\
A \vee \neg A &\Leftrightarrow w && \text{(Tautologie)}\\
A \wedge \neg A &\Leftrightarrow f && \text{(Widerspruch)}\\
A \vee f &\Leftrightarrow A\\
A \vee w &\Leftrightarrow w\\
A \wedge f &\Leftrightarrow f\\
A \wedge w &\Leftrightarrow A
\end{aligned}$$

Beweis ‚Implikation':

A	B	$\neg A$	$\neg A \vee B$	$A \Rightarrow B$	$(\neg A \vee B) \Leftrightarrow (A \Rightarrow B)$
w	w	f	w	w	w
w	f	f	f	f	w
f	w	w	w	w	w
f	f	w	w	w	w

© FH Technikum Wien

Die Folgerung (Schließen, Inferenz)

- Ziel von logik-basierten Ansätzen:
  aus bestehendem Wissen neues Wissen abzuleiten
- D.h. aus Wissensbasis WB (ev. umfangreiche aussagenlogische Formel) folgt eine Formel Q
- D.h. in jeder Welt, in der WB wahr ist, ist auch Q wahr, kurz:
  Immer wenn WB wahr ist, ist auch Q wahr

Definition 2.6
Eine Formel Q **folgt** aus einer Formel WB, wenn jedes Modell von WB auch ein Modell von Q ist. Man schreibt dann $^{WB} \models Q$.

© FH Technikum Wien

Die Folgerung (Schließen, Inferenz)

- Die Folgerung WB $\models$ Q ist eine semantische (metasprachliche) Aussage über **zwei Formeln** WB und Q:
  „Jedes Modell von WB ist ein Modell von Q“
- Dagegen ist WB $\Rightarrow$ Q **eine** Formel (die Implikation $\Rightarrow$ ist ein logischer Operator als Teil der Formel)

- Beispiel:

$$A \wedge B \models A \vee B$$

aber

$$A \vee B \not\models A \wedge B$$

© FH Technikum Wien

Berechenbarkeit / Komplexität

- **Komplexität:**
  Check auf Erfüllbarkeit von aussagenlogischen Formeln i.A. NP-vollständig (worst case), praktische Anwendungen fallen aber zum Glück meistens nicht in den worst case.

- **Wahrheitstafeln:**
  Nicht dabei hängen bleiben, bloß weil einfach. Werden zwar verwendet, aber nicht glauben man kommt damit alleine aus:
  - Logik ist viel mehr, zentral sind ist z.B. die **Folgerung**
  - In der Praxis sind Wahrheitstafeln auch viel zu ineffizient, Logikprogrammiersysteme verwenden sie daher **nicht**
  - Auch für erweiterte Logik (z.B. Prädikatenlogik) **nicht** anwendbar

© FH Technikum Wien

Anwendungen

- Aussagenlogische Beweiser in der Digitaltechnik (Verifikation digitaler Schaltungen)
- Binäre Entscheidungsbäume in der Mustererkennung
- Logikbasierte Expertensysteme

© FH Technikum Wien

2. Prädikatenlogik

© FH Technikum Wien

Prädikatenlogik: Mächtiger Lösungsansatz

Ermöglich Aussagen abhängig von Parametern, etwa

$$\forall u \, \forall v \;\; Steht_weiter_rechts(u, v) \;\Leftrightarrow$$

$$\exists x_u \, \exists y_u \, \exists x_v \, \exists y_v \;\; Position(u, x_u, y_u) \wedge Position(v, x_v, y_v) \wedge \; x_u > x_v$$

(Notation: „für alle u“, „es existiert ein x_u“)

© FH Technikum Wien

Motivation

- Aussagenlogik in vielen Bereichen ausreichend, aber…
- Gitter mit 100 x 100 Positionen,
  auf der sich 10 Roboter befinden können:

  „Roboter_7_befindet_sich_an_xy-Position_(35,79)“

- Wäre mit Aussagenlogik schwer auszudrücken
  (braucht 100 x 100 x 10 = 100.000 Variablen)

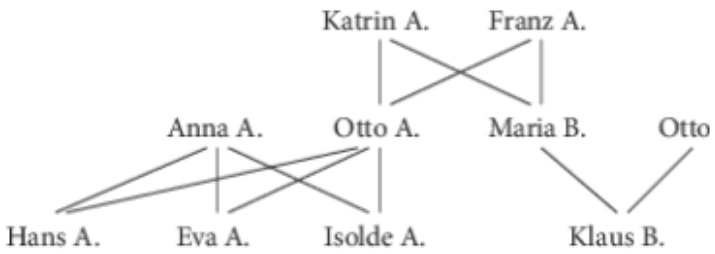
© FH Technikum Wien

Beispiele

Formel	Beschreibung
$\forall x\ frosch(x) \Rightarrow grün(x)$	Alle Frösche sind grün
$\forall x\ frosch(x) \wedge braun(x) \Rightarrow groß(x)$	Alle braunen Frösche sind groß
$\forall x\ mag(x, kuchen)$	Jeder mag Kuchen
$\neg\forall x\ mag(x, kuchen)$	Nicht jeder mag Kuchen
$\neg\exists x\ mag(x, kuchen)$	Keiner mag Kuchen
$\exists x\ \forall y\ mag(y, x)$	Es gibt etwas, das jeder mag
$\exists x\ \forall y\ mag(x, y)$	Es gibt jemanden, der alles mag
$\forall x\ \exists y\ mag(y, x)$	Jedes Ding wird von jemandem geliebt
$\forall x\ \exists y\ mag(x, y)$	Jeder mag etwas
$\forall x\ kunde(x) \Rightarrow mag(bob, x)$	Bob mag jeden Kunden
$\exists x\ kunde(x) \wedge mag(x, bob)$	Es gibt einen Kunden, der Bob mag
$\exists x\ bäcker(x) \wedge \forall y\ kunde(y) \Rightarrow mag(x, y)$	Es gibt einen Bäcker, der all seine Kunden mag
$\forall x\ älter(mutter(x), x)$	Jede Mutter ist älter als ihr Kind
$\forall x\ älter(mutter(mutter(x)), x)$	Jede Großmutter ist älter als das Kind ihrer Tochter
$\forall x\ \forall y\ \forall z\ rel(x, y) \wedge rel(y, z) \Rightarrow rel(x, z)$	*rel* ist eine transitive Relation

© FH Technikum Wien

Beispiel: Stammbaum



Rekursive Definition von Nachkomme:

$$\forall x\, \forall y\; nachkomme(x,y) \Leftrightarrow \exists z\; kind(x,y,z) \vee (\exists u\; \exists v\; kind(x,u,v) \wedge nachkomme(u,y)).$$

© FH Technikum Wien

Anwendungen

- Theorembeweiser in der Mathematik
- Expertensysteme
- Automatische Programmverifikation
- Semantic Web

© FH Technikum Wien

3. Logikorientierte Programmierung

© FH Technikum Wien

Logikorientierte Programmierung

- Durch ein Logikprogramm wird Wissen beschrieben. Der **Interpreter** (oder **Reasoner**, **Inferenz-Engine**) des Logikprogramms schließt dann darauf, was aus diesem Wissen folgt.

- Keine falschen Erwartungen haben:
  1. Logikorientierte Sprachen in der Regel nicht als Universalsprache konzipiert. Sie wollen keine Alternative zu OOP (C, C++, Java, etc.) sein, sondern eine Ergänzung (z.B. als Backend).
  2. Bekannte Konstrukte wie If-Verzweigungen, Schleifen, Prozeduren, etc. gibt es nicht. Auch spielt bei vielen Semantiken nicht einmal die Reihenfolge der Regeln eine Rolle (es gibt keine Abarbeitung von oben nach unten, sondern das Programm muss ganzheitlich erfüllt sein).

→ Logikorientierte Sprachen sind daher eher mit Abfragesprachen wie SQL zu vergleichen als mit prozeduraler Programmierung/OOP.

Logikorientierte Programmierung

- Beschreibung des Wissens verwendet spezielle Formeln der PL1 der folgenden Form, sogenannte **Regeln**:
  $B_1 \wedge \ldots \wedge B_m \wedge \neg B_{m+1} \wedge \ldots \wedge \neg B_n \Rightarrow H_1 \vee \ldots \vee H_k$

- Informell: Wenn alle `B's` (der *Regelrumpf*) wahr sind, dann muss auch zumindest eines der `H's` (der *Regelkopf*) wahr sein

- **Syntax in der Praxis** (ASCII-basiert)**:**

  H_1 `v` … `v` H_k `:-` B_1`,` …`,` B_m`,` `not` B_{m+1}`,` …`,` `not` B_n`.`

  oder (systemabhängig)

  H_1 `|` … `|` H_k `:-` B_1`,` …`,` B_m`,` `not` B_{m+1}`,` …`,` `not` B_n`.`

  wobei die `B's` und `H's` Atome der Form `p(`x_1`,` …`,` x_1`)` sind.

Logikorientierte Programmierung

- **Namenskonvention** in den meisten Systemen:
  Prädikate `p` sind klein geschriebene Strings
  Argumente `x` sind
  entweder *Konstanten* (klein geschrieben oder in " eingeschlossen)
  oder *Variablen* (groß geschrieben)

- **Konvention:**
  wenn `k=0` („leerer" Regelkopf) steht für false.
  Die gesamte Regel steht daher für

$$B_1 \wedge \ldots \wedge B_m \wedge \neg B_{m+1} \wedge \ldots \wedge \neg B_n \Rightarrow \bot$$

  und wird **Constraint** genannt.
  Bedeutung: der Regelrumpf darf nie wahr werden (sonst wäre die Implikation nicht erfüllt).

Logikorientierte Programmierung

Terminologie: bestimmte Regeltypen haben Sondernamen

- **Fakt** wenn `n=0` und `k=1`
  Beispiel: `employee(sue).`
  (etwas ist wahr ohne Vorbedingung)
- **disjunktiver Fakt** wenn `n=0` und `k>0`
  Beispiel: `cloudy|sunny|rainy.`
  (eine von mehreren Optionen ist wahr ohne Vorbedingung)
- **Constraint** wenn `k=0`
  Beispiel: `:- married(X), single(X).`
  (der leere Regelkopf steht implizit für false; drückt aus dass der Rumpf niemals wahr werden darf)

Logikorientierte Programmierung

Ein Logikprogramm besteht aus zwei Komponenten:

- **Intensionale Datenbank (IDB): Allgemeine** Problembeschreibung
- **Extensionale Datenbank (EDB):** Konkrete **Instanz** durch Fakten

- **Beispiel:**
  Berechnung des transitiven Abschlusses der Kanten in einem Graphen.

```
IDB={edge(Y, Z) :- edge(X, Y), edge(Y, Z).}
EDB={edge(a, b). edge(c, d), edge(u, v).}
```

- **Beachte:**
  Die Trennung hat rein praktischen Nutzen (z.B. Aufteilung in zwei Dateien). Formal bilden beide zusammen **eine** Menge von Regeln!

Logikorientierte Programmierung

Intuition:

Regeln drücken von rechts nach links gelesen eine Implikation aus

z.B. `edge(Y, Z) :- edge(X, Y), edge(Y, Z).`

„Wenn (X,Y) eine edge ist, und (Y,Z) ebenso, dann ist auch (Y,Z) eine edge“

Beachte:

- `edge` ist **kein** Schlüsselwort sondern ein Prädikat!
  Es könnte genauso gut `kante(X,Y)` oder `blafoo(X,Y)` heißen.
- Tatsächlich gibt es hier keine Schlüsselwörter. **Kein** Prädikat hat irgendeine inhärente Bedeutung in der Sprache, sinnvolle Bezeichnungen dienen nur der Verständlichkeit durch die User.
- Trotz Formulierung „Wenn … dann“ ist eine Implikation was anderes als eine If-Verzweigung in der prozeduralen Programmierung! Insbesondere wird dadurch kein zeitlicher Ablauf definiert: die Regeln werden nicht in einer bestimmten Reihenfolge abgearbeitet, eine Lösung muss die Regeln bloß insgesamt erfüllen.

Logikorientierte Programmierung

- **Beispiel: Reachability**

  IDB:

```
reach(X, Y) :- conn(X, Y).
conn(X, Z) :- conn(X, Y), conn(Y, Z).
```

  EDB: Menge von Fakten über Prädikat `conn`, z.B.

```
conn(heiligenstadt, spittelau).
conn(spittelau, friedensbrücke). …
```

- **Beispiel: Familienstammbaum (nur IDB)**

```
mother(M, C) :- childOf(C, M), female(M).
father(F, C) :- childOf(C, F), male(F).
sibling(C, S) :- childOf(X, C), childOf(X, S).
cousin(C1, C2) :- childOf(C1,P1), childOf(C2,P2),
                  sibling(P1,P2).
```

Logikprogrammiersysteme

- **Semantik**
  - Es gibt in der Praxis nicht **die** logikorientierte Programmierung, sondern verschiedene Semantiken (ähnlich wie unterschiedliche prozedurale Programmiersprachen).
  - Für jede Semantik gibt es mitunter mehrere Implementierungen, die teilweise leichte Unterschiede vorweisen oder proprietäre Erweiterungen anbieten
    (vgl. mehrere Compiler für C, mehrere Java-VMs).

- Wir verwenden die **Answer Set-Semantik (Antwortmengenprogrammierung)** als Beispiel und deren Implementierung im System **clingo**.
  Alles bisher Gesagte gilt unter anderem auch dafür.
- Alternativen (z.B. Prolog, kanren-Paket für Python, etc.) unterscheiden sich im Detail mehr oder weniger stark, basieren aber auf ähnlichen Grundideen.

Logikprogrammiersung nach der Answer Set Semantik

Idee:

- Ein **Programm** ist eine Menge von Regeln obiger Art.
- Es hat 0 bis mehrere (endlich viele) Lösungen in Form von sogenannten **Answer Sets** (=Antwortmengen).
- Ein **Answer Set** ist eine Menge von wahren Atomen (vgl. Interpretationen in Aussagenlogik), die
  1. alle Regeln erfüllt, und
  2. minimal*) ist.

  Minimalität stellt sicher, dass nichts grundlos auf wahr gesetzt wird.

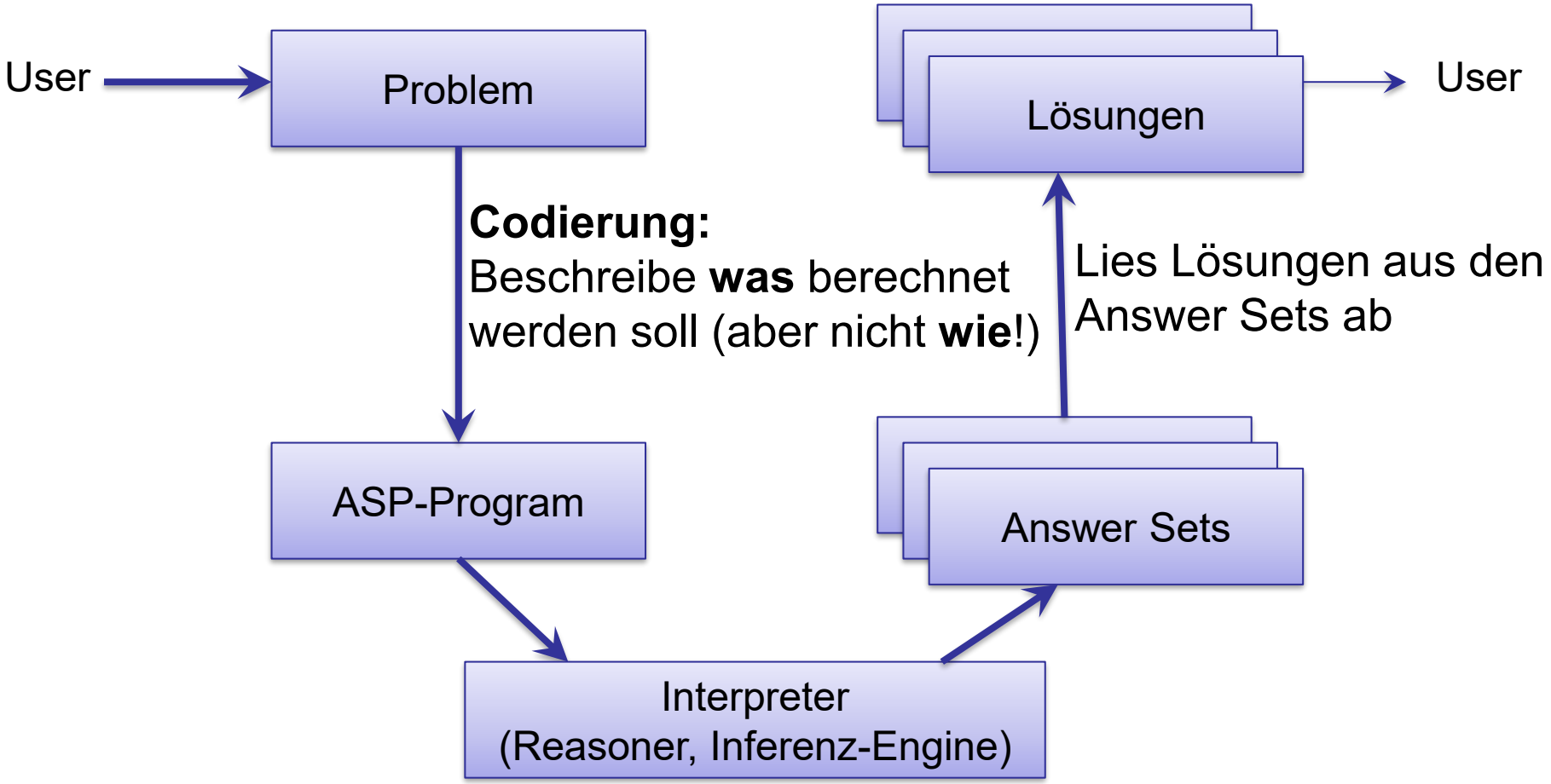
(* exakte Definition ist etwas komplexer, geht aber über diesen Kurs hinaus)

Beispiel für Minimalität

```
single :- not married.
married :- not single.
```

Obwohl `{single}`, `{married}` und `{single,married}` jeweils beide Regeln erfüllen, sind nur die ersten beiden Lösungen weil die letzte nicht minimal ist.

Logikprogrammiersung nach der Answer Set Semantik



System clingo

- Wir verwenden in der Übung clingo:
  https://potassco.org/ (Teil der Potassco-Suite)

  → Open-Source und derzeit am weitesten verbreitetes System unter der Answer Set Semantik

  → Lauffähig unter Windows, OS X, und Linux

- Manual:
  https://github.com/potassco/guide/releases/download/v2.2.0/guide.pdf
  Enthält viele **Beispiele**!

- Video-Tutorials der clingo-Entwickler:
  https://www.youtube.com/c/Potassco-live/videos
  (empfohlen sind in diesem Kurs vor allem die älteren Basics-Tutorials)

System clingo

- **Aufruf (command-line):**

  ```
  clingo -n 0 program.asp
  ```

  wobei `program.asp` eine Textdatei mit dem Logikprogramm ist (Dateiendung ist irrelevant, üblich sind: .dl, .lp oder .asp)

  Programm kann zur Strukturierung auch in mehrere Dateien gesplittet werden (es sind jedoch alle Regeln gleichwertig und verhalten sich als wären sie in einer Datei).

- **Wichtig:**
  clingo gibt standardmäßig nur die **erste** gefundene Lösung aus!
  Option `-n <N>` erlaubt die Berechnung von `<N>` Lösungen (z.B. `-n 5`) oder `-n 0` für alle

System clingo

- **Example:**

Anaconda Powershell Prompt

```
(base) PS C:\Users\Redl\Nextcloud\Redl\Courses FH Technikum\Knowledge Management\ASP-Examples> clingo -n 5 .\sudoku\sudoku-enc.dl
clingo version 5.4.0
Reading from .\sudoku\sudoku-enc.dl
Solving...
Answer: 1
assign(1,2,2) assign(1,1,7) assign(1,3,8) assign(2,2,6) assign(2,3,3) assign(3,2,9) assign(3,3,1) assign(2,1,4) assign(3,1,5) assign(1,4,3) assign(1,5,4) assign(1,6,5) assign(2,5,9) assign(2,6,1) assign(3,5,6) assign(3,6,8) a
ssign(2,4,2) assign(3,4,7) assign(1,7,1) assign(1,8,9) assign(1,9,6) assign(2,8,7) assign(2,9,5) assign(3,8,3) assign(3,9,2) assign(2,7,8) assign(3,7,4) assign(4,2,1) assign(4,1,2) assign(4,3,9) assign(5,2,7) assign(5,3,5) as
sign(6,2,4) assign(6,3,6) assign(5,1,8) assign(6,1,3) assign(4,5,7) assign(4,4,8) assign(4,6,6) assign(5,5,1) assign(5,6,3) assign(6,5,2) assign(6,6,9) assign(5,4,4) assign(6,4,5) assign(4,8,4) assign(4,7,5) assign(4,9,3) ass
ign(5,8,6) assign(5,9,9) assign(6,8,1) assign(6,9,8) assign(5,7,2) assign(6,7,7) assign(7,1,1) assign(7,2,3) assign(7,3,2) assign(8,2,8) assign(8,3,7) assign(9,2,5) assign(9,3,4) assign(8,1,9) assign(9,1,6) assign(7,5,8) assi
gn(7,4,9) assign(7,6,7) assign(8,5,5) assign(8,6,4) assign(9,5,3) assign(9,6,2) assign(8,4,6) assign(9,4,1) assign(7,8,5) assign(7,7,6) assign(7,9,4) assign(8,8,2) assign(8,9,1) assign(9,8,8) assign(9,9,7) assign(8,7,3) assig
n(9,7,9)
Answer: 2
assign(1,2,2) assign(1,1,7) assign(1,3,8) assign(2,2,9) assign(2,3,3) assign(3,2,6) assign(3,3,1) assign(2,1,4) assign(3,1,5) assign(1,4,3) assign(1,5,4) assign(1,6,5) assign(2,5,6) assign(2,6,1) assign(3,5,9) assign(3,6,8) a
ssign(2,4,2) assign(3,4,7) assign(1,7,1) assign(1,8,9) assign(1,9,6) assign(2,8,7) assign(2,9,5) assign(3,8,3) assign(3,9,2) assign(2,7,8) assign(3,7,4) assign(4,2,1) assign(4,1,2) assign(4,3,9) assign(5,2,7) assign(5,3,5) as
sign(6,2,4) assign(6,3,6) assign(5,1,8) assign(6,1,3) assign(4,5,7) assign(4,4,8) assign(4,6,6) assign(5,5,1) assign(5,6,3) assign(6,5,2) assign(6,6,9) assign(5,4,4) assign(6,4,5) assign(4,8,4) assign(4,7,5) assign(4,9,3) ass
ign(5,8,6) assign(5,9,9) assign(6,8,1) assign(6,9,8) assign(5,7,2) assign(6,7,7) assign(7,1,1) assign(7,2,3) assign(7,3,2) assign(8,2,8) assign(8,3,7) assign(9,2,5) assign(9,3,4) assign(8,1,9) assign(9,1,6) assign(7,5,8) assi
gn(7,4,9) assign(7,6,7) assign(8,5,5) assign(8,6,4) assign(9,5,3) assign(9,6,2) assign(8,4,6) assign(9,4,1) assign(7,8,5) assign(7,7,6) assign(7,9,4) assign(8,8,2) assign(8,9,1) assign(9,8,8) assign(9,9,7) assign(8,7,3) assig
n(9,7,9)
Answer: 3
assign(1,2,2) assign(1,1,7) assign(1,3,8) assign(2,2,6) assign(2,3,3) assign(3,2,9) assign(3,3,1) assign(2,1,4) assign(3,1,5) assign(1,4,3) assign(1,5,4) assign(1,6,5) assign(2,5,9) assign(2,6,1) assign(3,5,6) assign(3,6,8) a
ssign(2,4,2) assign(3,4,7) assign(1,7,1) assign(1,8,9) assign(1,9,6) assign(2,8,7) assign(2,9,5) assign(3,8,3) assign(3,9,2) assign(2,7,8) assign(3,7,4) assign(4,2,1) assign(4,1,2) assign(4,3,9) assign(5,2,4) assign(5,3,6) as
sign(6,2,7) assign(6,3,5) assign(5,1,8) assign(6,1,3) assign(4,5,7) assign(4,4,8) assign(4,6,6) assign(5,5,2) assign(5,6,3) assign(6,5,1) assign(6,6,9) assign(5,4,5) assign(6,4,4) assign(4,8,4) assign(4,7,5) assign(4,9,3) ass
ign(5,8,1) assign(5,9,9) assign(6,8,6) assign(6,9,8) assign(5,7,7) assign(6,7,2) assign(7,1,1) assign(7,2,3) assign(7,3,2) assign(8,2,8) assign(8,3,7) assign(9,2,5) assign(9,3,4) assign(8,1,9) assign(9,1,6) assign(7,5,8) assi
gn(7,4,9) assign(7,6,7) assign(8,5,5) assign(8,6,4) assign(9,5,3) assign(9,6,2) assign(8,4,6) assign(9,4,1) assign(7,8,5) assign(7,7,6) assign(7,9,4) assign(8,8,2) assign(8,9,1) assign(9,8,8) assign(9,9,7) assign(8,7,3) assig
n(9,7,9)
Answer: 4
assign(1,2,2) assign(1,1,7) assign(1,3,8) assign(2,2,6) assign(2,3,3) assign(3,2,9) assign(3,3,1) assign(2,1,4) assign(3,1,5) assign(1,4,3) assign(1,5,4) assign(1,6,5) assign(2,5,9) assign(2,6,1) assign(3,5,6) assign(3,6,8) a
ssign(2,4,2) assign(3,4,7) assign(1,7,1) assign(1,8,9) assign(1,9,6) assign(2,8,7) assign(2,9,5) assign(3,8,3) assign(3,9,2) assign(2,7,8) assign(3,7,4) assign(4,2,1) assign(4,1,2) assign(4,3,9) assign(5,2,4) assign(5,3,6) as
sign(6,2,7) assign(6,3,5) assign(5,1,8) assign(6,1,3) assign(4,5,7) assign(4,4,8) assign(4,6,6) assign(5,5,2) assign(5,6,3) assign(6,5,1) assign(6,6,9) assign(5,4,5) assign(6,4,4) assign(4,8,4) assign(4,7,5) assign(4,9,3) ass
ign(5,8,1) assign(5,9,7) assign(6,8,6) assign(6,9,8) assign(5,7,9) assign(6,7,2) assign(7,1,1) assign(7,2,3) assign(7,3,2) assign(8,2,8) assign(8,3,7) assign(9,2,5) assign(9,3,4) assign(8,1,9) assign(9,1,6) assign(7,5,8) assi
gn(7,4,9) assign(7,6,7) assign(8,5,5) assign(8,6,4) assign(9,5,3) assign(9,6,2) assign(8,4,6) assign(9,4,1) assign(7,8,5) assign(7,7,6) assign(7,9,4) assign(8,8,2) assign(8,9,1) assign(9,8,8) assign(9,9,9) assign(8,7,3) assig
n(9,7,7)
Answer: 5
assign(1,2,2) assign(1,1,7) assign(1,3,8) assign(2,2,6) assign(2,3,3) assign(3,2,9) assign(3,3,1) assign(2,1,4) assign(3,1,5) assign(1,4,3) assign(1,5,4) assign(1,6,5) assign(2,5,9) assign(2,6,1) assign(3,5,6) assign(3,6,8) a
ssign(2,4,2) assign(3,4,7) assign(1,7,1) assign(1,8,9) assign(1,9,6) assign(2,8,7) assign(2,9,5) assign(3,8,3) assign(3,9,2) assign(2,7,8) assign(3,7,4) assign(4,2,1) assign(4,1,2) assign(4,3,9) assign(5,2,4) assign(5,3,6) as
sign(6,2,7) assign(6,3,5) assign(5,1,8) assign(6,1,3) assign(4,5,7) assign(4,4,8) assign(4,6,6) assign(5,5,3) assign(5,6,2) assign(6,5,1) assign(6,6,9) assign(5,4,5) assign(6,4,4) assign(4,8,4) assign(4,7,5) assign(4,9,3) ass
ign(5,8,1) assign(5,9,9) assign(6,8,6) assign(6,9,8) assign(5,7,7) assign(6,7,2) assign(7,1,1) assign(7,2,3) assign(7,3,2) assign(8,2,8) assign(8,3,7) assign(9,3,4) assign(8,1,9) assign(9,1,6) assign(7,5,8) assi
gn(7,4,9) assign(7,6,7) assign(8,5,5) assign(8,6,4) assign(9,5,2) assign(9,6,3) assign(8,4,6) assign(9,4,1) assign(7,8,5) assign(7,7,6) assign(7,9,4) assign(8,8,2) assign(8,9,1) assign(9,8,8) assign(9,9,7) assign(8,7,3) assig
n(9,7,9)
SATISFIABLE

Models       : 5+
Calls        : 1
Time         : 0.051s (Solving: 0.03s 1st Model: 0.00s Unsat: 0.00s)
CPU Time     : 0.031s
(base) PS C:\Users\Redl\Nextcloud\Redl\Courses FH Technikum\Knowledge Management\ASP-Examples>
```

Eckpunkte des Moduls

© FH Technikum Wien

- ***Aussagenlogik*** als erster (ältester) Formalismus, um eine Wissensdatenbank aufzubauen und logische Schlussfolgerungen darauf anzuwenden.

- ***Syntaktische Korrektheit*** sowie ***semantische Korrektheit und Vollständigkeit*** gefordert, allerdings in vielen Fällen nicht berechenbar (vgl. Compiler).

- ***Prädikatenlogik*** mächtiger als Aussagenlogik.

- ***Logikorientierte Programmierung*** am Beispiel einer konkreten Semantik und des Systems clingo.

© FH Technikum Wien